

Samurai Fight 开发指导文档

横版2D像素风1v1格斗游戏 - 开发指南

目录

- 一、游戏概述
- 二、产品方案
- 三、技术方案
- 四、项目结构
- 五、核心系统设计
- 六、数据配置
- 七、开发路线图
- 八、开发规范

一、游戏概述

1.1 基本信息

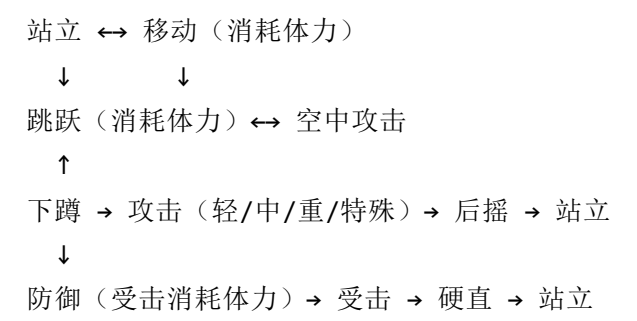
- 游戏名称：Samurai Fight（武士对决）
- 游戏类型：横版2D像素风格1v1格斗游戏
- 开发语言：C++
- 图形库：SFML 3.0
- 构建系统：CMake
- 目标帧率：60 FPS
- 分辨率：1280×720

1.2 核心玩法

两名玩家分别操控各自角色，通过移动、跳跃、攻击、防御等动作进行对战。角色受到攻击会进入硬直状态并损失生命值。防御时可减免伤害且不会硬直，但会消耗体力。当一方生命值归零或倒计时结束时，生命值多的一方获胜。采用三局两胜制。

动作	描述	可取消性	体力消耗
跳跃	向上跳跃	起跳后不可取消	消耗 10
下蹲	降低受击判定	可随时站起	无消耗
轻攻击	伤害低，速度快	前摇可取消	无消耗
中攻击	伤害中，速度中	前摇可取消	无消耗
重攻击	伤害高，速度慢	前摇不可取消	无消耗
特殊技	个别角色独有	角色特定	消耗 15
防御	减免伤害，不硬直	后摇不可取消	受击时消耗
受击硬直	被击中后无法操作	无法取消	无消耗

2.2.4 状态转移图



2.3 战斗系统

2.3.1 攻击属性

攻击类型	伤害倍率	前摇帧	判定帧	后摇帧	可取消	取消窗口
轻攻击	1.0x	3帧	2帧	5帧	是	2-5帧
中攻击	1.5x	5帧	3帧	8帧	是	4-8帧
重攻击	2.0x	8帧	4帧	12帧	否	-
特殊技	1.8x	角色特定	角色特定	角色特定	角色特定	角色特定

2.3.2 连招系统

输入缓冲窗口：12 帧（0.2秒）

简单连招示例

- 轻攻击 → 轻攻击 → 中攻击
- 轻攻击 → 重攻击
- 中攻击 → 中攻击 → 重攻击
- 下 + 攻击 → 特殊技（角色特定）

2.3.3 防御与体力

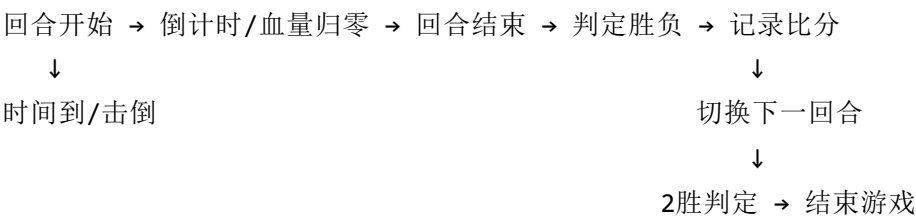
- 防御时受到伤害减免 50%
- 防御时体力消耗 = 受到伤害值 × 0.5
- 体力耗尽时防御失效，受到全额伤害并硬直
- 移动时体力耗尽：移动速度降低 50%

2.3.4 碰撞系统

- 角色碰撞框**：用于角色间碰撞
- 攻击判定框**：用于攻击命中检测
- 受击判定框**：用于判断受击区域
- 使用矩形碰撞框（AABB）

2.4 回合制系统

2.4.1 流程



2.4.2 规则

- 每回合默认 60 秒
- 三局两胜制
- 时间结束时，生命值多的一方获胜
- 时间结束且生命值相等，判为平局，双方各计 0 胜

2.5 UI 界面系统

2.5.1 界面列表

1. 主菜单

- 开始游戏
- 设置
- 退出游戏

2. 角色选择界面

- 两个角色选择框（1P在左，2P在右）
- 角色预览动画
- 角色名称和属性预览
- 确认按钮

3. 对战界面

- 顶部：血条、体力槽、计时器、比分
- 中间：战斗场景
- 底部：（可选）连招提示

4. 暂停菜单

- 继续游戏
- 设置
- 退出到主菜单

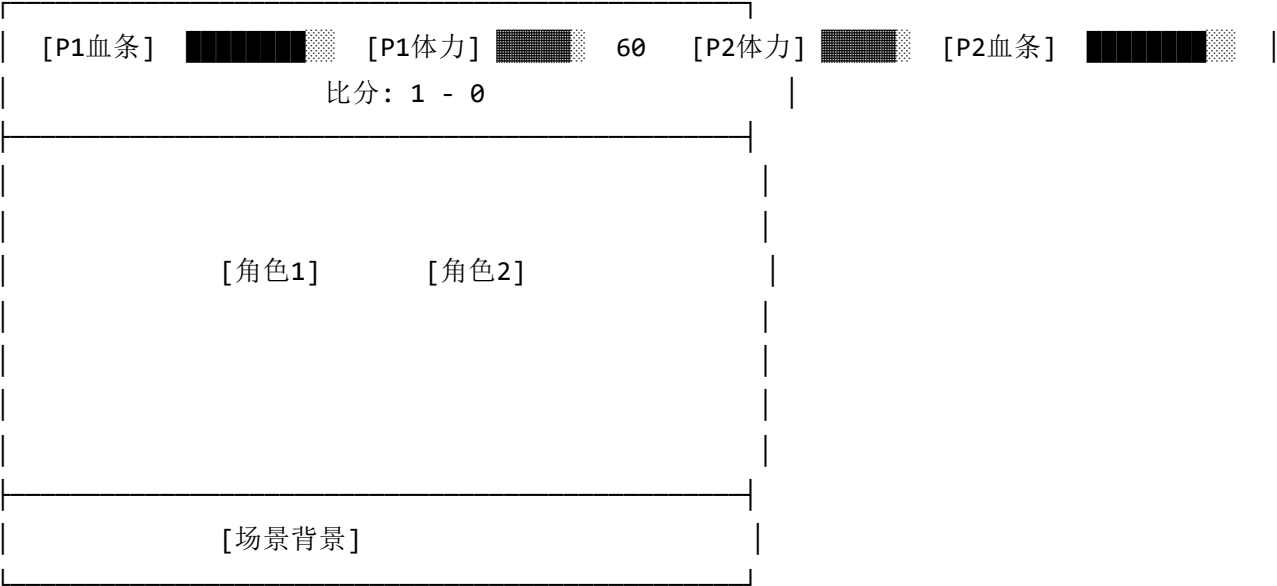
5. 设置界面

- 音量控制（主音量、BGM、音效）
- 按键配置（1P/2P分别设置）
- 画面设置（窗口/全屏、VSync）
- 返回按钮

6. 结果界面

- 胜利者展示
- 比分显示
- 重赛按钮
- 返回角色选择
- 返回主菜单

2.5.2 UI布局（1280×720）



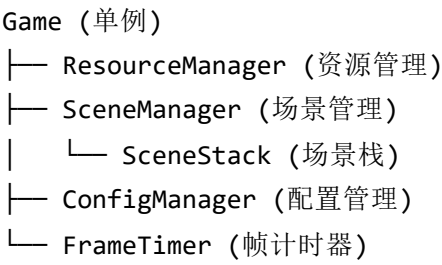
三、技术方案

3.1 技术栈

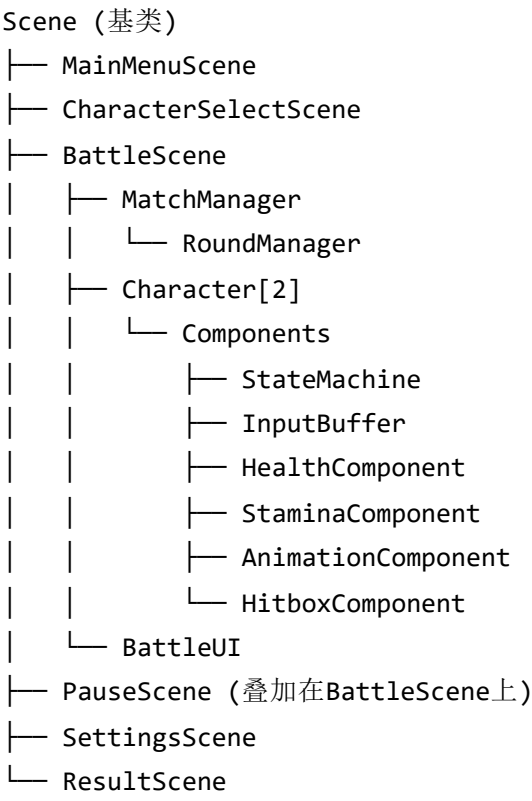
类别	技术选型
编程语言	C++17
图形库	SFML 3.0
构建系统	CMake 3.16+
JSON解析	nlohmann/json
版本控制	Git
像素分辨率	角色 128×128，窗口 1280×720

3.2 架构设计

3.2.1 核心架构

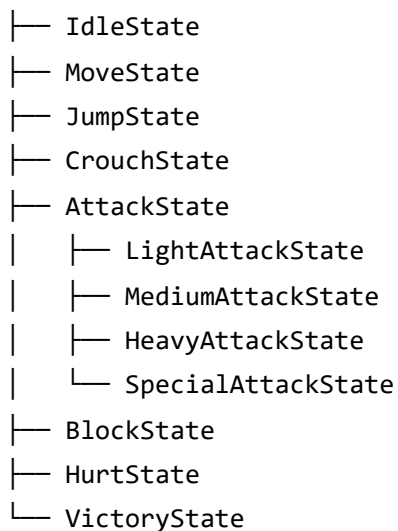


3.2.2 场景架构



3.2.3 状态机架构

CharacterState (基类)



3.3 关键系统

3.3.1 帧逻辑系统

```
// 60 FPS 帧同步
class FrameTimer {
    static constexpr int TARGET_FPS = 60;
    static constexpr float FRAME_TIME = 1.0f / 60.0f;

    void update();           // 更新累积时间
    bool shouldUpdate();    // 判断是否应该执行逻辑更新
    void decreaseAccumulator(); // 消耗一帧时间
};
```

帧更新流程


```

void Game::run() {
    while (window->isOpen()) {
        frameTimer.update();

        while (frameTimer.shouldUpdate()) {
            handleEvents();          // 处理输入
            update(FRAME_TIME);      // 游戏逻辑更新
            frameTimer.decreaseAccumulator();
        }

        render(); // 渲染
    }
}

```

3.3.2 输入缓冲系统

```

// 输入缓冲窗口：12帧
class InputBuffer {
    static constexpr int BUFFER_SIZE = 12;

    void pushInput(const InputCommand& command);
    bool hasInput(InputAction action, int frameWindow);
    std::vector<InputCommand> getRecentInputs(int count);
};

```

3.3.3 1P/2P 帧分离

```
void BattleScene::update(float deltaTime) {  
    // 1. 收集输入  
    for (int i = 0; i < 2; i++) {  
        inputBuffers[i].collectInput();  
    }  
  
    // 2. 分别更新每个角色的帧逻辑  
    for (int i = 0; i < 2; i++) {  
        updateCharacterFrame(i);  
    }  
  
    // 3. 判定碰撞  
    checkCollisions();  
  
    // 4. 处理结果  
    processResults();  
}
```

四、项目结构

```
SamuraiFight/
├─ CMakeLists.txt
├─ README.md
├─ DEVELOPMENT_GUIDE.md
├─ assets/
│   └─ characters/
│       └─ musashi/
│           ├── config.json
│           ├── idle.png
│           ├── walk.png
│           ├── jump.png
│           ├── attack_light.png
│           ├── attack_medium.png
│           ├── attack_heavy.png
│           ├── attack_special.png
│           ├── block.png
│           ├── hurt.png
│           └─ crouch.png
│       └─ ronin/
│           ├── config.json
│           └─ ...
│   └─ backgrounds/
│       ├── dojo.png
│       ├── forest.png
│       ├── castle.png
│       ├── sunset.png
│       ├── night.png
│       └─ winter.png
│   └─ ui/
│       ├── buttons/
│           ├── button_normal.png
│           ├── button_hover.png
│           └─ button_pressed.png
│       ├── bars/
│           ├── health_bar_bg.png
│           ├── health_bar_fill.png
│           ├── stamina_bar_bg.png
│           └─ stamina_bar_fill.png
│       └─ icons/
│           └─ timer_icon.png
```

```

|       └─ win_icon.png
|   └─ fonts/
|       └─ pixel_font.ttf
|   └─ sounds/
|       └─ hit/
|           └─ hit_light.wav
|           └─ hit_medium.wav
|           └─ hit_heavy.wav
|       └─ block/
|           └─ block.wav
|       └─ footsteps/
|           └─ footstep.wav
|       └─ ui/
|           └─ menu_select.wav
|           └─ menu_confirm.wav
|           └─ countdown.wav
|       └─ round/
|           └─ round_start.wav
|           └─ round_end.wav
|           └─ victory.wav
|   └─ music/
|       └─ menu_theme.ogg
|       └─ battle_theme.ogg
|   └─ config/
|       └─ game_config.json
|       └─ keybindings.json
└─ src/
    └─ main.cpp
    └─ core/
        └─ Game.cpp
        └─ Game.h
        └─ GameLoop.cpp
        └─ GameLoop.h
        └─ FrameTimer.cpp
        └─ FrameTimer.h
        └─ Constants.h
    └─ managers/
        └─ ResourceManager.cpp
        └─ ResourceManager.h
        └─ SceneManager.cpp
        └─ SceneManager.h
        └─ ConfigManager.cpp
        └─ ConfigManager.h

```

```
| |─ scenes/
| | |─ Scene.cpp
| | |─ Scene.h
| | |─ MainMenuScene.cpp
| | |─ MainMenuScene.h
| | |─ CharacterSelectScene.cpp
| | |─ CharacterSelectScene.h
| | |─ BattleScene.cpp
| | |─ BattleScene.h
| | |─ PauseScene.cpp
| | |─ PauseScene.h
| | |─ SettingsScene.cpp
| | |─ SettingsScene.h
| | |─ ResultScene.cpp
| | |─ ResultScene.h
| |─ entities/
| | |─ Entity.cpp
| | |─ Entity.h
| | |─ Character.cpp
| | |─ Character.h
| | |─ CharacterData.h
| | |─ CharacterFactory.cpp
| | |─ CharacterFactory.h
| |─ components/
| | |─ StateMachine.cpp
| | |─ StateMachine.h
| | |─ InputBuffer.cpp
| | |─ InputBuffer.h
| | |─ HealthComponent.cpp
| | |─ HealthComponent.h
| | |─ StaminaComponent.cpp
| | |─ StaminaComponent.h
| | |─ AnimationComponent.cpp
| | |─ AnimationComponent.h
| | |─ HitboxComponent.cpp
| | |─ HitboxComponent.h
| |─ states/
| | |─ CharacterState.cpp
| | |─ CharacterState.h
| | |─ IdleState.cpp
| | |─ IdleState.h
| | |─ MoveState.cpp
| | |─ MoveState.h
```

```
| | | └─ JumpState.cpp
| | | └─ JumpState.h
| | | └─ CrouchState.cpp
| | | └─ CrouchState.h
| | | └─ AttackState.cpp
| | | └─ AttackState.h
| | | └─ BlockState.cpp
| | | └─ BlockState.h
| | | └─ HurtState.cpp
| | | └─ HurtState.h
| | | └─ VictoryState.cpp
| | | └─ VictoryState.h
| └─ input/
| | └─ InputManager.cpp
| | └─ InputManager.h
| | └─ InputCommand.h
| | └─ KeyBindingManager.cpp
| | └─ KeyBindingManager.h
| └─ combat/
| | └─ ComboSystem.cpp
| | └─ ComboSystem.h
| | └─ HitboxManager.cpp
| | └─ HitboxManager.h
| | └─ CombatCalculator.cpp
| | └─ CombatCalculator.h
| └─ battle/
| | └─ RoundManager.cpp
| | └─ RoundManager.h
| | └─ MatchManager.cpp
| | └─ MatchManager.h
| | └─ BattleUI.cpp
| | └─ BattleUI.h
| └─ ui/
| | └─ UIElement.cpp
| | └─ UIElement.h
| | └─ Button.cpp
| | └─ Button.h
| | └─ HealthBar.cpp
| | └─ HealthBar.h
| | └─ StaminaBar.cpp
| | └─ StaminaBar.h
| | └─ Timer.cpp
| | └─ Timer.h
```

```
| | | └─ ScoreDisplay.cpp
| | | └─ ScoreDisplay.h
| | └─ utils/
| |   └─ JsonLoader.cpp
| |   └─ JsonLoader.h
| |   └─ MathUtils.h
| |   └─ DebugUtils.h
└─ include/
```

五、核心系统设计

5.1 键位配置（方案A）

5.1.1 默认键位

玩家 1（键盘左侧）

动作	按键
向左	A
向右	D
跳跃	W
下蹲	S
轻攻击	J
中攻击	K
重攻击	L
特殊技	U
防御	I
暂停	Escape

玩家 2（键盘右侧）

动作	按键
向左	方向键左 ←
向右	方向键右 →
跳跃	方向键上 ↑
下蹲	方向键下 ↓
轻攻击	小键盘 1
中攻击	小键盘 2
重攻击	小键盘 3
特殊技	小键盘 4
防御	小键盘 5
暂停	Escape（共用）

5.1.2 键位自定义

- 玩家可在设置界面重新映射键位
- 键位配置保存至 `assets/config/keybindings.json`
- 提供"恢复默认键位"按钮

5.2 JSON 配置格式

5.2.1 角色配置

```
{
  "id": "musashi",
  "name": "武藏",
  "description": "平衡型武士，擅长剑术",
  "stats": {
    "maxHp": 100,
    "maxStamina": 100,
    "moveSpeed": 3.0,
    "jumpHeight": 400.0,
    "staminaRecovery": 0.5,
    "moveStaminaCost": 0.1,
    "jumpStaminaCost": 10.0
  },
  "attacks": {
    "light": {
      "damage": 10,
      "startupFrames": 3,
      "activeFrames": 2,
      "recoveryFrames": 5,
      "stunFrames": 8,
      "staminaCost": 0,
      "canCancel": true,
      "cancelWindow": {"start": 2, "end": 5}
    },
    "medium": {
      "damage": 15,
      "startupFrames": 5,
      "activeFrames": 3,
      "recoveryFrames": 8,
      "stunFrames": 12,
      "staminaCost": 0,
      "canCancel": true,
      "cancelWindow": {"start": 4, "end": 8}
    },
    "heavy": {
      "damage": 20,
      "startupFrames": 8,
      "activeFrames": 4,
      "recoveryFrames": 12,
```

```
    "stunFrames": 18,
    "staminaCost": 0,
    "canCancel": false
  },
  "special": {
    "enabled": true,
    "name": "居合斩",
    "damage": 18,
    "startupFrames": 10,
    "activeFrames": 5,
    "recoveryFrames": 15,
    "stunFrames": 20,
    "staminaCost": 15,
    "canCancel": false,
    "input": ["down", "attack_heavy"]
  }
},
"combos": [
  {
    "inputs": ["light", "light", "medium"],
    "damageMultiplier": 1.2,
    "name": "基础三连"
  },
  {
    "inputs": ["light", "heavy"],
    "damageMultiplier": 1.1,
    "name": "轻重连击"
  }
],
"animations": {
  "idle": {
    "spriteSheet": "characters/musashi/idle.png",
    "frameCount": 4,
    "frameDuration": 8,
    "loop": true
  },
  "walk": {
    "spriteSheet": "characters/musashi/walk.png",
    "frameCount": 6,
    "frameDuration": 6,
    "loop": true
  },
  "jump": {
```

```
    "spriteSheet": "characters/musashi/jump.png",
    "frameCount": 4,
    "frameDuration": 4,
    "loop": false
  },
  "attack_light": {
    "spriteSheet": "characters/musashi/attack_light.png",
    "frameCount": 8,
    "frameDuration": 2,
    "loop": false
  },
  "attack_medium": {
    "spriteSheet": "characters/musashi/attack_medium.png",
    "frameCount": 10,
    "frameDuration": 2,
    "loop": false
  },
  "attack_heavy": {
    "spriteSheet": "characters/musashi/attack_heavy.png",
    "frameCount": 12,
    "frameDuration": 2,
    "loop": false
  },
  "attack_special": {
    "spriteSheet": "characters/musashi/attack_special.png",
    "frameCount": 15,
    "frameDuration": 2,
    "loop": false
  },
  "block": {
    "spriteSheet": "characters/musashi/block.png",
    "frameCount": 2,
    "frameDuration": 4,
    "loop": true
  },
  "hurt": {
    "spriteSheet": "characters/musashi/hurt.png",
    "frameCount": 3,
    "frameDuration": 4,
    "loop": false
  },
  "crouch": {
    "spriteSheet": "characters/musashi/crouch.png",
```

```

        "frameCount": 2,
        "frameDuration": 8,
        "loop": true
    }
},
"hitboxes": {
    "idle": [
        {"x": 40, "y": 20, "width": 48, "height": 100, "type": "hurtbox"}
    ],
    "attack_light": [
        {"x": 40, "y": 20, "width": 48, "height": 100, "type": "hurtbox"},
        {"x": 88, "y": 40, "width": 40, "height": 30, "type": "hitbox", "activeFrames": [3, 4]}
    ]
}
}

```

5.2.2 游戏配置

```

{
    "window": {
        "width": 1280,
        "height": 720,
        "title": "Samurai Fight",
        "fullscreen": false,
        "vsync": true
    },
    "gameplay": {
        "targetFPS": 60,
        "roundTime": 60,
        "winsNeeded": 2
    },
    "audio": {
        "masterVolume": 1.0,
        "bgmVolume": 0.8,
        "sfxVolume": 1.0
    }
}

```

5.2.3 键位配置

```
{
  "player1": {
    "moveLeft": "A",
    "moveRight": "D",
    "jump": "W",
    "crouch": "S",
    "attackLight": "J",
    "attackMedium": "K",
    "attackHeavy": "L",
    "attackSpecial": "U",
    "block": "I"
  },
  "player2": {
    "moveLeft": "Left",
    "moveRight": "Right",
    "jump": "Up",
    "crouch": "Down",
    "attackLight": "Numpad1",
    "attackMedium": "Numpad2",
    "attackHeavy": "Numpad3",
    "attackSpecial": "Numpad4",
    "block": "Numpad5"
  },
  "global": {
    "pause": "Escape"
  }
}
```

5.3 场景背景

计划 6 个背景场景：

1. **道场 (dojo)** - 室内，木质地板，日式建筑
2. **竹林 (forest)** - 户外，竹林，阳光透过
3. **城堡 (castle)** - 夜晚，日本城堡剪影
4. **夕阳 (sunset)** - 黄昏，橙色天空，山景
5. **夜晚 (night)** - 夜晚，月光，樱花飘落
6. **雪景 (winter)** - 冬天，雪花飘落，雪地

六、数据配置

6.1 初始角色

角色 1：武藏 (Musashi)

- 定位：平衡型
- 特点：攻守兼备，适合新手
- 特殊技：居合斩（下 + 重攻击）
- 优势：均衡的属性，稳定的连招

角色 2：浪人 (Ronin)

- 定位：速度型
- 特点：移动快，攻击快，但伤害低
- 特殊技：闪避斩（下 + 重攻击）
- 优势：高机动性，快速连击

6.2 属性对比

属性	武藏	浪人
生命值	100	90
体力值	100	110
移动速度	3.0	3.5
跳跃高度	400	420
体力恢复	0.5/帧	0.6/帧
轻攻击伤害	10	8
中攻击伤害	15	12
重攻击伤害	20	16
特殊技伤害	18	15

七、开发路线图

阶段 1：项目框架（第 1 周）

- ☐ CMake 项目搭建
- ☐ Game 类、GameLoop 实现
- ☐ ResourceManager 基础实现
- ☐ SceneManager 框架
- ☐ 主菜单场景

交付物：可运行的主菜单窗口

阶段 2：角色系统（第 2-3 周）

- ☐ Character 类实现
- ☐ 状态机框架
- ☐ 基础状态（Idle, Move, Jump, Crouch）
- ☐ AnimationComponent
- ☐ JSON 角色数据加载
- ☐ HitboxComponent

交付物：可在场景中移动、跳跃的角色

阶段 3：战斗系统（第 4-5 周）

- ☐ AttackState 实现
- ☐ 输入缓冲系统
- ☐ 连招系统
- ☐ 碰撞检测
- ☐ 体力槽系统
- ☐ 防御系统
- ☐ HurtState 实现

交付物：可进行基本对战的游戏

阶段 4：回合与比赛（第 6 周）

- ☐ RoundManager
- ☐ MatchManager
- ☐ 回合过渡动画

- ☐ 胜负判定
- ☐ VictoryState

交付物：完整的回合制对战

阶段 5：UI 系统（第 7 周）

- ☐ HealthBar, StaminaBar
- ☐ Timer, ScoreDisplay
- ☐ 角色选择界面
- ☐ 暂停菜单
- ☐ 结果界面

交付物：完整的游戏UI

阶段 6：设置与配置（第 8 周）

- ☐ 设置界面
- ☐ 键位重映射
- ☐ 音量控制
- ☐ 配置文件保存/加载

交付物：完整的设置系统

阶段 7：打磨与优化（第 9 周）

- ☐ 音效集成
- ☐ 背景场景实现
- ☐ 视觉效果优化
- ☐ 性能优化
- ☐ Bug 修复

交付物：可发布的游戏

八、开发规范

8.1 代码规范

8.1.1 命名约定

- **类名**：PascalCase（如 Character，GameState）
- **函数名**：camelCase（如 updatePosition，takeDamage）
- **变量名**：camelCase（如 currentHealth，moveSpeed）
- **常量**：UPPER_SNAKE_CASE（如 MAX_HEALTH，TARGET_FPS）
- **成员变量**：使用 m_ 前缀（如 m_currentHealth）
- **静态变量**：使用 s_ 前缀（如 s_instance）

8.1.2 文件组织

- 每个类一个 .h 和 .cpp 文件
- 头文件使用 #pragma once 防护
- 实现文件包含对应头文件

8.1.3 注释规范

```
/**
 * @brief 简短描述
 *
 * 详细描述
 *
 * @param paramName 参数描述
 * @return 返回值描述
 */
```

8.2 Git 提交规范

8.2.1 提交消息格式

<type>(<scope>): <subject>

<body>

<footer>

type 类型:

- feat : 新功能
- fix : Bug 修复
- docs : 文档更新
- style : 代码格式
- refactor : 重构
- test : 测试
- chore : 构建/工具

示例:

feat(combat): add combo system

- Implement input buffer for combo detection
- Add combo data loading from JSON
- Support 3-hit combo for musashi

Closes #123

8.3 分支策略

main (稳定版本)

```
├─ develop (开发分支)
│   ├── feature/character-system
│   ├── feature/combat-system
│   ├── feature/ui-system
│   └─ bugfix/input-buffer-issue
└─ release/v1.0
```

8.4 测试策略

8.4.1 单元测试

- 使用 Catch2 或 Google Test
- 测试核心逻辑 (状态机、输入缓冲、碰撞检测)

8.4.2 手动测试清单

☐ 所有角色动作是否正常

- ☐ 连招是否正确触发
- ☐ 体力槽是否正确消耗/恢复
- ☐ 回合计时是否准确
- ☐ 胜负判定是否正确
- ☐ UI 是否正确显示
- ☐ 键位重映射是否保存
- ☐ 所有场景是否正常切换

8.5 性能指标

- 帧率：稳定 60 FPS
- 内存：< 500 MB
- 启动时间：< 3 秒
- 场景切换：< 1 秒

九、附录

9.1 SFML 3.0 迁移注意事项

SFML 3.0 相比 2.x 有重大变化：

1. 事件处理

```
// SFML 2.x
sf::Event event;
while (window.pollEvent(event)) { ... }

// SFML 3.0
while (const std::optional event = window.pollEvent()) { ... }
```

2. 纹理加载

```
// SFML 2.x
sf::Texture texture;
texture.loadFromFile("path");

// SFML 3.0
auto texture = sf::Texture::loadFromFile("path").value();
```

3. 绘制

```
// SFML 2.x
window.draw(sprite);

// SFML 3.0
window.draw(sprite); // 相同
```

9.2 常见问题

Q: 角色动画不流畅?

A: 检查帧计时器是否正确实现，确保每帧时间固定为 1/60 秒。

Q: 碰撞检测不准确?

A: 检查碰撞框数据是否正确配置，确保使用角色的实际位置而非锚点。

Q: 输入延迟?

A: 确保在事件处理后立即更新输入缓冲，不要在渲染前才处理输入。

Q: 连招无法触发?

A: 检查输入缓冲窗口是否足够大（建议 12 帧），确保取消窗口配置正确。

9.3 参考资源

- [SFML 3.0 文档](#)
- [nlohmann/json 文档](#)
- [格斗游戏设计模式](#)
- [帧数据基础](#)

文档版本：1.0

最后更新：2026年6月18日

维护者：lllhhh